

OpenAI API Prompt

Summarizing and editing text

- **API Endpoints:** You discovered that endpoints are specific access points for different services provided by an API, similar to how different doors in a hospital lead to various departments.
- **API Authentication:** You learned that accessing most API services requires authentication, typically through a unique key.
- **Costs Associated with API Use:** It was highlighted that using OpenAI's API services incurs costs based on the model and the size of the input and output, though the course exercises are configured to avoid these costs.
- **Navigating API Documentation:** The importance of consulting API documentation for details on endpoints, their functionalities, and authentication setup was emphasized.
- **Creating an API Request:** You practiced creating a request to the OpenAI API using Python, specifically targeting the completions endpoint to complete a text prompt. The process involves specifying the model and the prompt in the request.

Here's a snippet of the code you worked with:

```
# Create the OpenAI client
client = OpenAI(api_key="<OPENAI_API_TOKEN>")

# Create a request to the Completions endpoint
response = client.completions.create(
    model="gpt-3.5-turbo-instruct",
    prompt="Who developed ChatGPT?"
)
```

Chap 1: Prompt Engineering with the OpenAI API

Engineering Prompts

- **Action Verbs:** You discovered the importance of using specific action verbs like "write," "explain," and "describe" to direct the model's tasks clearly. Avoiding vague verbs ensures the model understands the expected action.
- **Precise Instructions:** Crafting prompts with detailed instructions about the context, output length, format, style, and audience leads to more accurate responses. For instance, specifying "describe the behavior and characteristics of Golden Retrievers" yields more focused results than a general prompt about dogs.

- **Output Length:** You learned to control the output length by specifying expectations directly in the prompt, such as requesting "two sentences" on a topic. This approach helps manage the model's output more effectively than using parameters like `max_tokens`, which might cut responses off.
- **Delimited Prompts:** The lesson introduced the use of delimiters (e.g., triple backticks) to structure prompts clearly, especially when including input data for tasks like text summarization. This technique helps the model identify and process the input correctly.

Using f-strings: You practiced embedding variables into prompts using Python's f-strings, allowing for dynamic prompt creation. For example:

`# Create a prompt that completes the story`

```
prompt = f"Complete the story delimited by triple backticks. ```{story}```"
```

Few-shot Prompting

- **Tables:** You discovered how to instruct a model to generate a table by clearly mentioning the expected columns. For example, asking for a table of movies for action lovers with columns for title and rating.
- **Lists:** You explored generating lists, such as the top five cities to visit, and learned the importance of specifying the format, like choosing between ordered or unordered lists.
- **Structured Paragraphs:** You saw how to request paragraphs with a specific structure, including clear headings and subheadings, by explicitly mentioning format requirements in the prompt.
- **Custom Formats:** You learned to request outputs in custom formats by breaking down the prompt into distinct parts, including defining input text and specifying how the output should be formatted.
- **Conditional Prompts:** You were introduced to conditional prompts that incorporate logic or conditions, following an if-else style to structure the prompt based on logical conditions.

For example, to generate a table of must-read sci-fi books, you used the following code:

`# Set your API key`

```
client = OpenAI(api_key="<OPENAI_API_TOKEN>")
```

`# Create a prompt that generates the table`

```
prompt = "Generate a table containing 10 books I should read if I am a sci-fi lover, with columns for Title, Author, and Year."
```

`# Get the response`

```
response = get_response(prompt)
```

```
print(response)
```

Multi-step prompting

- **Understanding Multi-step Prompting:** You learned that multi-step prompting involves dividing an end goal into a sequence of steps, making it easier for the model to produce the desired outcome. This is particularly useful for tasks requiring a series of actions or cognitive processing.
- **Applications:** You saw how multi-step prompts can be applied in various contexts, such as writing a coherent travel blog or analyzing the correctness of a Python code snippet. For example, to assess a code's correctness, you might start with checking syntax, followed by verifying functionality like handling division by zero.

Code Implementation:

```
# Set your API key
client = OpenAI(api_key="<OPENAI_API_TOKEN>")

# Create a prompt detailing steps to plan the trip
prompt = """
    Help me plan a beach vacation.
    Step 1 - Specify four potential locations for beach vacations
    Step 2 - State some accommodation options in each
    Step 3 - State activities that could be done in each
    Step 4 - Evaluate the pros and cons for each destination
    """

response = get_response(prompt)

print(response)
```

Chain-of-thought and Self-Consistency Prompting

- **Chain-of-thought prompting** requires the model to outline its reasoning steps before arriving at a final answer. This approach is especially useful for complex problems, as it breaks down the model's thought process into understandable segments. For instance, when solving a math problem, the model explains each step of its calculation, making its reasoning transparent and easier to follow.
- **Self-consistency prompting** involves generating multiple chain-of-thought responses and selecting the most common answer as the final output. This technique addresses the issue of potential errors in a single chain of thought by aggregating multiple reasoning paths, thereby increasing the likelihood of a correct answer.

Here's an example of how you might set up a chain-of-thought prompt in Python using the OpenAI API:

```
# Set your API key
client = OpenAI(api_key="<OPENAI_API_TOKEN>")
```

```
# Create the chain-of-thought prompt
prompt = "Compute the age of my friend's father in 10 years, given
that now he's double my friend's age, and my friend is 20. Give a
step by step explanation."

response = get_response(prompt)
print(response)
```

Text Transformation

- **Language Translation:** You discovered how to translate text from one language to another by specifying input and output languages in the prompt. For example, translating an English product description into French.
- **Tone Adjustment:** You explored how to adjust the tone of text to suit different business needs, such as changing a promotional message from informal to formal and persuasive.
- **Writing Improvement:** You learned how to use LLMs for correcting spelling, grammar, and punctuation, as well as enhancing clarity by restructuring text for better readability.

In the exercises, you applied these concepts by crafting prompts for:

Translating marketing messages into multiple languages, using a code snippet like:

```
client = OpenAI(api_key="<OPENAI_API_TOKEN>")
prompt = f"""Translate the English marketing message delimited by
triple backticks to French, Spanish, and Japanese
```{marketing_message}```
"""
response = get_response(prompt)
```

- Adjusting the tone of email marketing to be professional, positive, and user-centric.
- Improving writing by proofreading and adjusting the tone of user-generated content to be formal and friendly.

## Text Analysis

You learned about the application of prompt engineering in text analysis, focusing on text classification and entity extraction. Text analysis involves examining text to derive meaningful information, crucial for businesses to understand customer feedback, classify inquiries, and extract relevant data for decision-making.

- **Text Classification:** You discovered how to categorize text into predefined classes, such as sentiment analysis, where text is classified as positive, negative, or neutral. The

importance of specifying classification categories in the prompt was emphasized to improve accuracy. For example, when classifying a smartwatch review's sentiment, you would craft a prompt that asks the model to classify the sentiment as positive, negative, or neutral, expecting a one-word response.

- **Entity Extraction:** You explored how to extract specific entities like names, places, and dates from text. Effective entity extraction requires specifying the entities and the output format. An example involved extracting product name, display size, and camera resolution from a product description, formatting the response as an unordered list.

Additionally, you practiced crafting prompts for a customer support scenario, where you classified tickets into categories such as technical issues, billing inquiries, or product feedback. You also worked on extracting key entities from customer support tickets using a few-shot prompt approach to inform the model of the desired output format.

```
Craft a prompt to classify the ticket
prompt = f"""Classify the ticket delimited by triple backticks as
technical issue, billing inquiry, or product feedback. Your response
should just contain the class and nothing else.

```{ticket}```"""
```

Role-playing prompts for chatbots

- **Definition and Importance:** Understanding that role-playing prompts instruct chatbots to mimic a specific role, adjusting their language and behavior accordingly.
- **Symbolic Example:** Through an example with a chatbot assuming roles of a customer support agent, product manager, and sales engineer, you saw how the same query could elicit different responses based on the role played.
- **Implementation:** You learned to implement role-playing prompts by crafting a `system_prompt` that directs the chatbot to act in a defined role, such as a financial analyst or a technology journalist, to provide specialized responses.

Additionally, you practiced creating a role-playing `system_prompt` for a learning advisor chatbot, designed to recommend textbooks based on user queries about their background and goals. The exercise involved:

```
# Set your API key
client = OpenAI(api_key="<OPENAI_API_TOKEN>")
# Craft the system_prompt using the role-playing approach
system_prompt = "Act as a learning advisor who receives queries from
users mentioning their background, experience, and goals, and
accordingly provides a response that recommends a tailored learning
```

path of textbooks, including both beginner-level and more advanced options."

Incorporating external context

- **Why external context is crucial:** Chatbots, especially those built on pre-trained language models like OpenAI's GPT, often lack up-to-date information or specific knowledge outside their training data. Providing external context helps bridge this gap, allowing for more accurate and relevant responses.

Incorporating context through sample conversations: You saw how to use previous interactions as a way to inform the chatbot about specific topics or preferences. This method involves feeding the model with a series of user and assistant messages that outline the context needed for future queries.

For instance, to inform a chatbot about the types of items a delivery service offers, you might include a sample conversation where this information is discussed:

```
context_question = "What types of items can be delivered using  
MyPersonalDelivery?"
```

```
context_answer = "We deliver everything from everyday essentials such  
as groceries, medications, and documents to larger items like  
electronics, clothing, and furniture. However, please note that we  
currently do not offer delivery for hazardous materials or extremely  
fragile items requiring special handling."
```

Using system prompts for context: A more streamlined approach involves directly embedding the necessary context within the system prompt. This method is particularly effective for conveying concise, specific information about services, policies, or capabilities without the need for extensive back-and-forth examples.

You practiced creating a system prompt that included a detailed service description for a hypothetical delivery service, enabling the chatbot to answer queries based on this embedded information:

```
system_prompt = f"""\nYou are a customer service chatbot for\nMyPersonalDelivery whose service description is delimited by triple\nbackticks. You should respond to user queries in a gentle way.\n```\n{service_description}\n```"""
```